Week 5: Database design: ER diagrams and normalisation

From a description to a design

- **Entity** = a thing we store data about (student, CCA). It becomes a **table**.
- **Attribute** = a detail of an entity. It becomes a **column**.
- **Primary key (PK)** = identifies one row. Never NULL, never repeated.
- **Foreign key (FK)** = a column that holds another table's primary key.
- **Relationship** = how entities connect.

Types of relationship

Type	Example	In tables
One-to-one	Student : locker	FK on either side (unique)
One-to-many	Class : students	FK on the **many** side
Many-to-many	Students : CCAs	A **junction table** with two FKs

```
student_cca(student_id FK, cca_id FK, role)
PK = (student_id, cca_id)       -- a composite key
```

ER diagram (crow's foot)

```
STUDENT ||--o{ STUDENT_CCA }o--|| CCA
  one student has zero or more memberships
  one CCA has zero or more memberships
```

`||` exactly one · `o|` zero or one · `o{` zero or many · `|{` one or many.

Why not one big table? Anomalies

- **Update:** change a value in one row but not the other copies.
- **Insert:** cannot record a fact until another fact exists.
- **Delete:** removing one fact loses a different fact.
- Root cause: **redundancy**, the same fact stored many times.

Normalisation: the three normal forms

Form	Rule	Fix
1NF	Every cell holds **one** value; no repeating groups	Put each value in its own row (maybe a new table)
2NF	1NF, and no **partial dependency**: no column depends on only **part** of a composite key	Move it to a table keyed by that part
3NF	2NF, and no **transitive dependency**: no non-key column depends on another non-key column	Move both to their own table

> Rule of thumb: every non-key attribute must depend on **the key, the whole key, and nothing but the key**.

Worked example

```
ORDERS(order_no, item_code, item_name, qty)
PK (order_no, item_code)
item_name depends only on item_code -> 2NF
becomes:
  ORDER_LINE(order_no, item_code, qty)
  ITEM(item_code PK, item_name)
```

```
STUDENT(id, name, class, class_teacher)
class_teacher depends on class -> 3NF
becomes:
  STUDENT(id, name, class FK)
  CLASS(class PK, class_teacher)
```

Process

- List the nouns (entities) and their details (attributes).
- Find relationships; draw the ER diagram; resolve many-to-many.
- Choose primary keys; add foreign keys on the many side.
- Check 1NF, 2NF, 3NF.
- Do **not** store what you can calculate (age, totals).